

# **SVA timeline lite**

Assertions check protocol rules on the timeline, not just at end of test

# Overlap, next cycle, and vacuous

- Pipe arrow overlap means when a is true, check b in the same cycle
- Double pipe arrow non-overlap means when a is true, check b on the next cycle
- An attempt starts when the antecedent fires, a high on that clock edge
- Pass means the consequent matched; fail means a fired but b did not
- Vacuous success means a never went high
- Real SVA wraps this in a property and assert property on posedge clock

# Browser lab

Digital Design and Verification Platform

HomeTools

Home / Tools / SVA implication timeline

INTERACTIVE TOOL

## SVA implication timeline

Sketch a  $a \rightarrow b$  (overlap) and a  $a \Rightarrow b$  (next-cycle) on a short wave — pass, fail, or **vacuous** success. Not a full SVA engine. Starter: overlapping pass at cycle 2.

Starter example: a  $\rightarrow$  b overlapping pass — at cycle 2 both a and b are high.

Load starter example

Challenges 0 / 22 cleared

Quiz:  $\rightarrow$ : a  $\rightarrow$  b means...

☐ when a is true, b must be true in the same cycle

☐ b is checked two cycles later always

☐ a and b are wires in the DUT netlist only

☐ the assertion is disabled

Show hint

Check

Next

Idle

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

Core ideas

$\rightarrow$

Overlapping: check b in the same cycle as a.

$\Rightarrow$

Non-overlap: check b in the next cycle.

Attempt

Each cycles where a is true starts a check.

Vacuous

If a never rises, success is vacuous.

Lab

WAVE PRESET

IMPLICATION

SVA timeline lab starter

# Real UVM literacy

```
wsl - module15-sva-timeline/examples/sva-sketch

# Track A ~ real Unix
# real shell session (Track A ~ SVA implication timeline)

$ pwd
/mnt/d/proj/designs/learning/courses/learn_uvm2017/module15-sva-timeline

$ ls examples/sva-sketch/
implication.txt

$ cat examples/sva-sketch/implication.txt
SVA implication sketch (literacy)

Operators (lite):
  a  $\mapsto$  b   overlapping: when a true, b must be true SAME cycle
  a  $\mapsto$  b   non-overlapping: when a true, b must be true NEXT cycle

Typical SVA form:
property p_req_ack;
  @(posedge clk) req  $\mapsto$  ack;    // overlap example
endproperty
assert property (p_req_ack);

property p_req_ack_next;
  @(posedge clk) req  $\mapsto$  ack;    // next-cycle example
endproperty
assert property (p_req_ack_next);

Evaluation outcomes:
PASS    - antecedent fired and consequent matched
FAIL    - antecedent fired but consequent did not match
VACUOUS - antecedent never true + no failing attempt (weak check)

Example wave (cycle index 0..7):
clk:  _|_|_|_|_|_|_|_
a:    _|_|_|_|_|_|_|_ (a=1 @ cycle 2)
b:    _|_|_|_|_|_|_|_ (b=1 @ cycle 2 +  $\mapsto$  pass)
b:    _|_|_|_|_|_|_|_ (b=1 @ cycle 3 +  $\mapsto$  pass)

Attempt rule:
  implication check starts when a is true on a sampled edge

Where SVA lives:
  inline in RTL module
  bind assertion module to DUT
  runs in parallel with UVM testbench

SVA vs UVM scoreboard:
  SVA - cycle/signal-level protocol rules close to DUT
  SB  - transaction-level expect vs actual

Common mistakes:
  confuse  $\mapsto$  and  $\mapsto$ 
  treat vacuous pass as thorough verification
  forget @(posedge clk) sampling event

# SVA examples often live in RTL/bind - no dedicated legacy lab file in this course tree
# use implication.txt + sva-timeline browser lab for literacy
```

Real shell — SVA implication sketch

# Pitfalls to watch

- Do not confuse overlap and next-cycle operators, they check different cycles
- Do not treat vacuous pass as strong coverage, if a never fires, the rule never exercised
- Do not write assertions without clocking, posedge clk is the usual sampling event
- And remember

# Your turn

- Complete the checklist for at least one track, preferably both
- In the browser, load overlap fail and explain the verdict
- On paper, sketch a pipe arrow b pass and a double-pipe b pass on the same wave
- When you are ready